

Lección 0.1 - Probabilidad y Matemáticas

Lección 0.1 — Probabilidad y Matemáticas para Machine Learning

Tags: #fundamentos #matematicas #probabilidad #trucoai #futbolai

Duración estimada: 3 días

Prerrequisitos: Ninguno

Objetivo

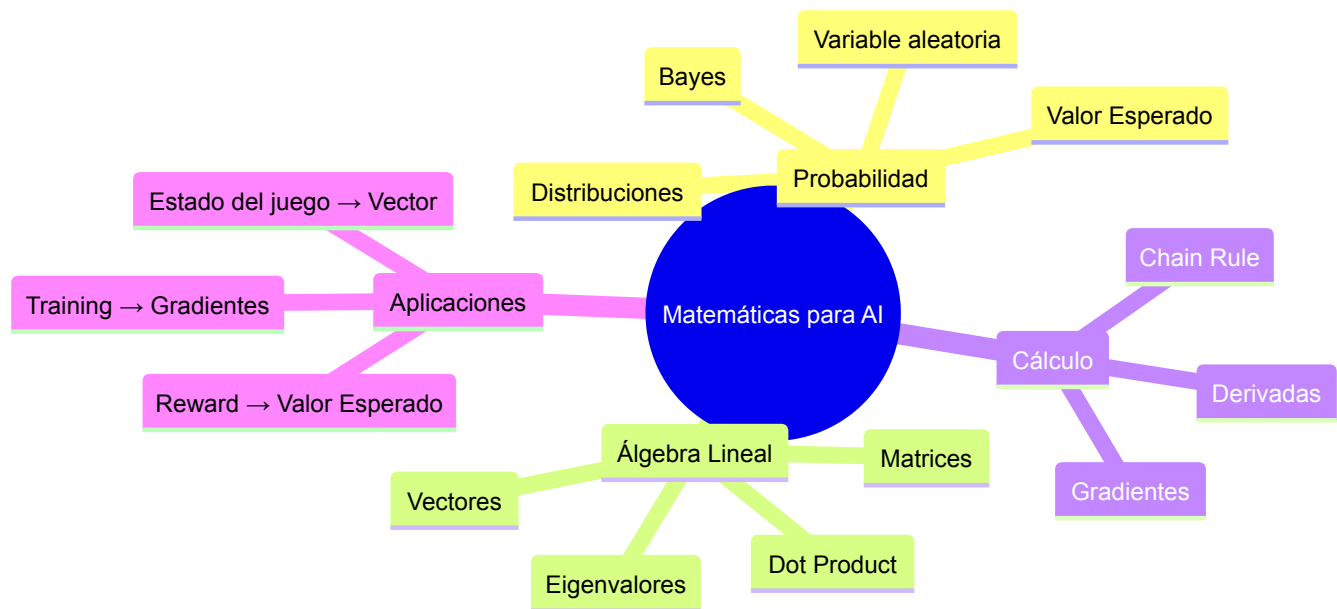
Al final de esta lección vas a poder entender —y explicarle a alguien más— los conceptos matemáticos que aparecen en TODOS los papers de Truco y Fútbol que tenemos. No necesitás ser matemático, necesitás **intuición**.

¿Por qué esto es importante para TrucoAI y FutbolAI?

Cada paper que leíste usa estas ideas:

Concepto	Dónde aparece en tus papers
Probabilidad	Cada mano de Truco es un evento aleatorio. CFR calcula probabilidades de ganar.
Distribuciones	Las cartas del oponente siguen una distribución de probabilidad (no sabemos exactamente cuáles tiene).
Valor esperado	"¿Convierto el Truco?" = calcular si el valor esperado de aceptar es positivo.
Matrices	El estado del juego se representa como vectores. Las redes neuronales son operaciones de matrices.
Gradientes	Cómo se entrena una red: siguiendo el gradiente hacia abajo.
Regresión	Predecir el valor de una posición en el campo de fútbol.

Diagrama de conceptos



1. PROBABILIDAD — El corazón del problema

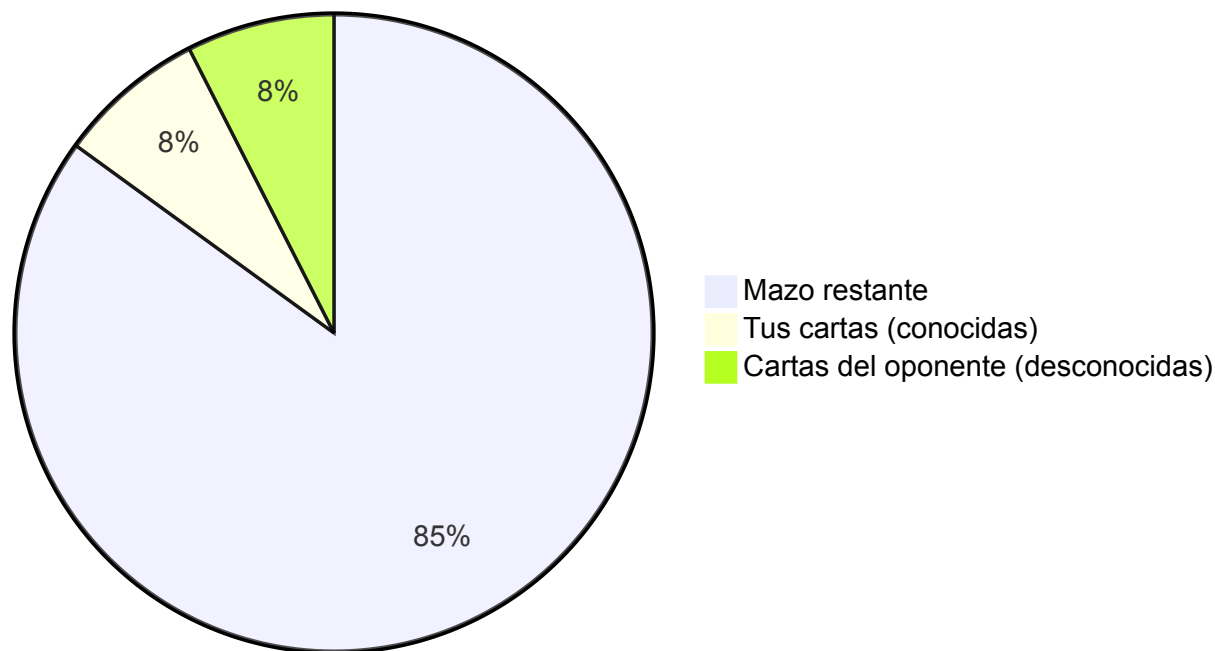
1.1 ¿Qué es una probabilidad?

Intuición: Si mezclás el mazo de Truco (40 cartas) y agarrás una, la probabilidad de que sea un 3 de espada es 1/40.

$$P(A) = (\text{casos favorables}) / (\text{casos totales})$$

En Truco: Cuando te dan 3 cartas, hay 37 cartas que NO ves. La probabilidad de que el oponente tenga el 3 de espada es 1/37. Cada decisión que tomás (jugar envideo, cantar truco) modifica tu **creencia** sobre lo que el otro tiene.

distribución de cartas en una mano de Truco



1.2 Variable aleatoria

Intuición: Una variable cuyo valor es... aleatorio. Como el resultado de "sacar una carta del mazo".

- **Discreta:** Solo puede tomar ciertos valores (ej: el palo de una carta $\rightarrow$ {espada, basto, oro, copa})
- **Continua:** Puede tomar cualquier valor (ej: la distancia al arco en un tiro de fútbol)

Visualización interactiva:

 Abre `Aprendizaje/Interactivos/probabilidad-distribuciones.html` en tu navegador para ver cómo cambian las distribuciones de probabilidad

1.3 Distribución de probabilidad

Intuición: Es una tabla que dice "qué tan probable es cada resultado posible".

Distribución uniforme (el mazo de Truco)

Carta:	1	2	3	4	5	6	7	10	11	12
Prob:	1/40	1/40	1/40	1/40	1/40	1/40	1/40	1/40	1/40	1/40

Cada carta tiene exactamente la misma probabilidad de salir.

Distribución de las cartas del oponente

Esto NO es uniforme. Si vos tenés el 3 de espada, el oponente **no puede** tenerlo. Esto se llama **probabilidad condicional**.

```
Error parsing Mermaid diagram!
```

```
Parse error on line 5:
```

```
...e]    B --> E[Los "fijos" cambian todo]
```

```
-----^
```

```
Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-)', 'STADIUMEND',  
'SUBROUTINEEND', 'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND', 'TRAPEND',  
'INVTRAPEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'STR'
```

1.4 Probabilidad Condicional — CLAVE para Truco

Intuición: La probabilidad de algo CAMBIA cuando tenés nueva información.

$P(A \mid B)$ = probabilidad de A DADO que pasó B

Ejemplo Truco:

- $P(\text{oponente tiene flor}) \approx 8\%$ al inicio
- $P(\text{oponente tiene flor} \mid \text{cantó envido y aceptó}) \approx 35\%$
- $P(\text{oponente tiene flor} \mid \text{cantó envido y se fue al mazo}) \approx 0.1\%$

Esto es Theory of Mind básico: el oponente revela información con sus acciones. Una subida al Truco te dice algo sobre su mano.

1.5 Teorema de Bayes

Intuición: Cómo actualizar tus creencias cuando ves nueva evidencia.

$$P(\text{creencia} \mid \text{evidencia}) = \frac{P(\text{evidencia} \mid \text{creencia}) \times P(\text{creencia})}{P(\text{evidencia})}$$

Para nosotros: Cuando un oponente canta Truco, la evidencia modifica nuestra creencia sobre su mano.

```
Error parsing Mermaid diagram!
```

```
Parse error on line 4:
```

```
...encia}    C --> D[P(buena mano | canta
```

```
-----^
```

```
Expecting 'SQE', 'DOUBLECIRCLEEND', 'PE', '-)', 'STADIUMEND',
```

```
'SUBROUTINEEND', 'PIPE', 'CYLINDEREND', 'DIAMOND_STOP', 'TAGEND', 'TRAPEND',  
'INVTRAPEND', 'UNICODE_TEXT', 'TEXT', 'TAGSTART', got 'PS'
```

1.6 Valor Esperado

Intuición: "En promedio, ¿cuánto voy a ganar o perder si hago esta jugada muchas veces?"

$$E[X] = \sum (\text{valor} \times \text{probabilidad})$$

El cálculo que define al Truco:

Envido:

- Aceptar: 60% de ganar 2 puntos $\times$ 60% = 1.2 puntos esperados
40% de perder 2 puntos $\times$ 40% = -0.8 puntos
VALOR ESPERADO = +0.4 puntos 
- No aceptar: 0 puntos (seguro)

En FútbolAI: La recompensa esperada de patear al arco vs hacer un pase.

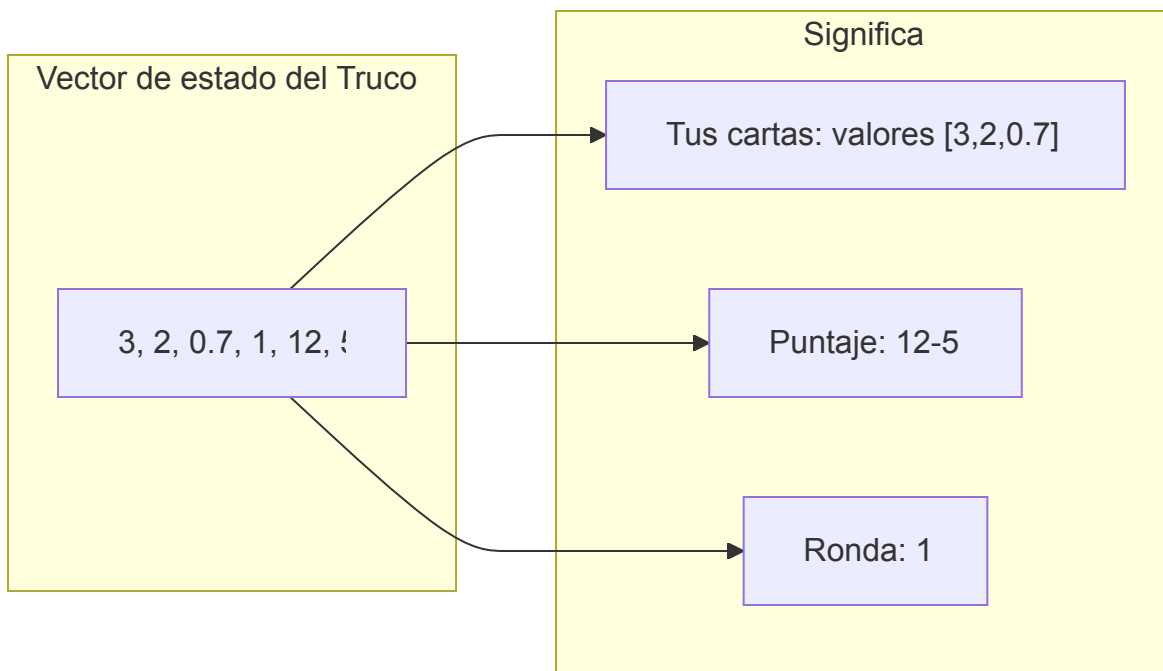
Patear: 15% gol (+1) + 40% despeje (0) + 45% pierde pelota (-0.1) = +0.15 - 0.045 = +0.105 

Pasar: 30% llega al compañero (+0.05) + 70% pierde (0) = +0.015

2. ÁLGEBRA LINEAL — Cómo piensa una red neuronal

2.1 Vectores

Intuición: Una lista de números. Es la forma en que una computadora representa "cosas".

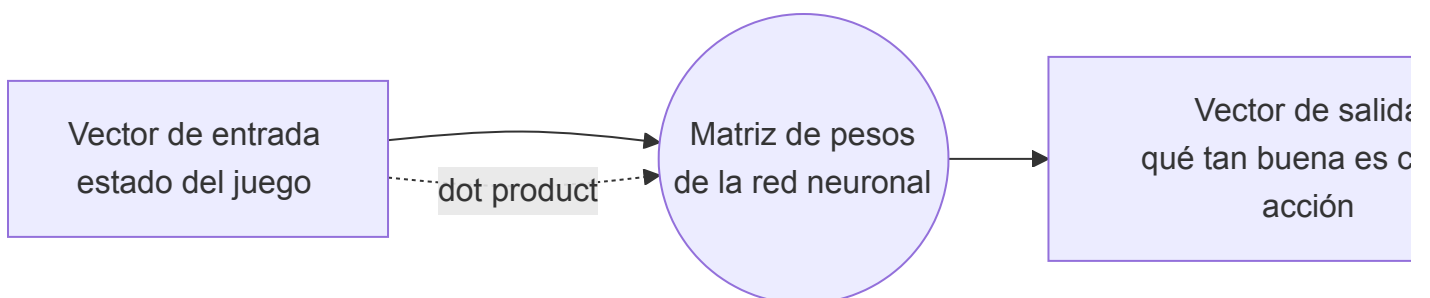


Un vector real para Truco:

```
[mano_player=[3, 2, 7], # valores de tus 3 cartas
palo_player=[1, 3, 2], # palos (1=espada, 2=basto...)
score_propio=12,
score_oponente=5,
ronda_actual=1,
ronda_1_resultado=1,      # ganaste/ perdiste / empate
enviado_aceptado=1,
truco_en_juego=0,
cartas_jugadas=[...]      # 40 posiciones, 1 si ya salió
]
```

2.2 Matrices y Dot Product

Intuición: Una matriz es una tabla de números. El dot product mide "qué tanto se parecen" dos vectores.



Fórmula mágica de una neurona:

$$\begin{aligned} \text{salida} &= W \cdot x + b \\ &= (w_1 \times x_1 + w_2 \times x_2 + \dots + w_n \times x_n) + b \end{aligned}$$

Donde:

- x = vector de entrada (el estado del juego)
- W = matriz de pesos (lo que la red "aprendió")
- b = bias (un ajuste)
- Resultado = qué tan buena es cada acción posible

2.3 ¿Por qué necesitamos esto?

Deep CFR (el paper de Noam Brown que está en tu carpeta de Truco) usa **redes neuronales** para aproximar el comportamiento de CFR en juegos enormes. La red recibe un vector de estado y devuelve:

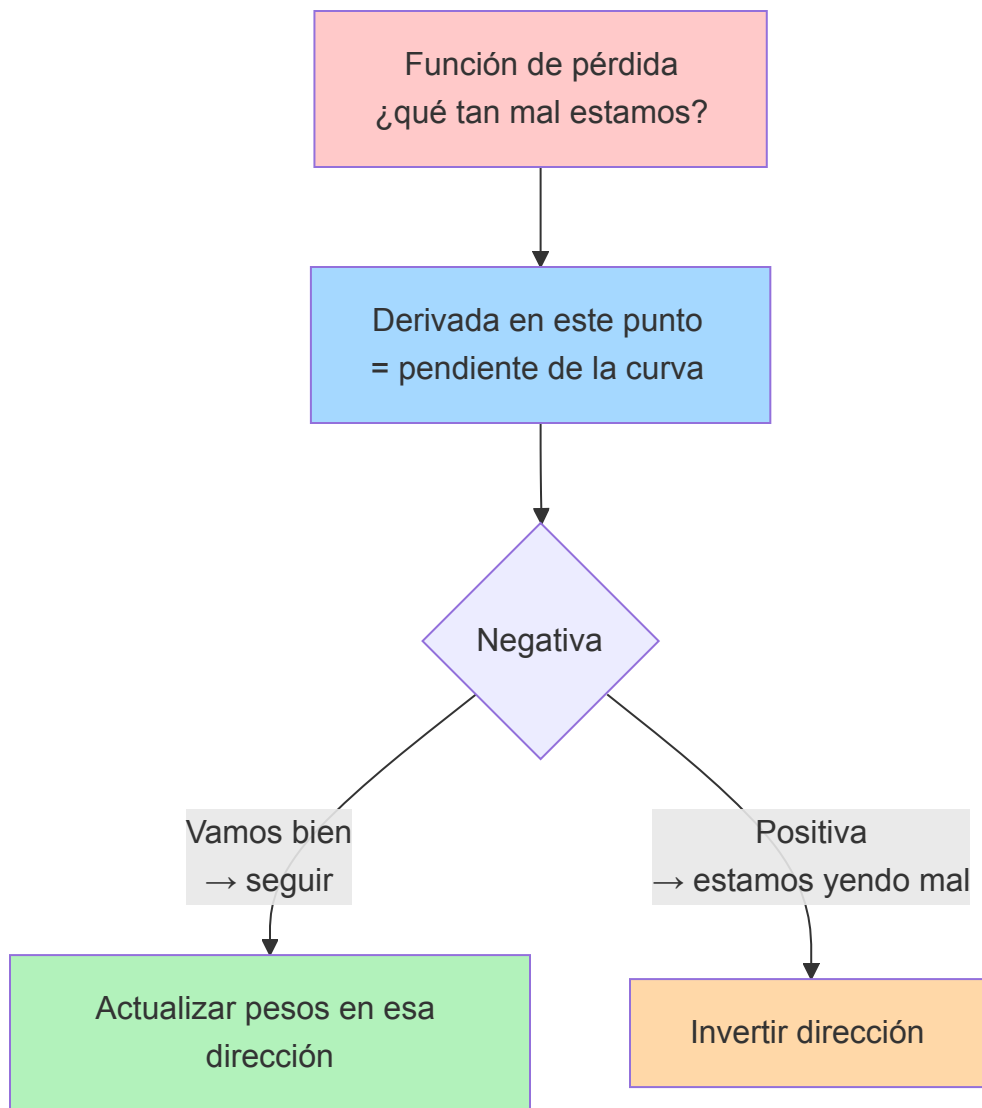
- **Policy network:** qué carta jugar
- **Value network:** qué valor tiene cada estado

Sin álgebra lineal, no podés entender cómo funciona ni una sola neurona de esas redes.

3. CÁLCULO DIFERENCIAL — Cómo aprende una red

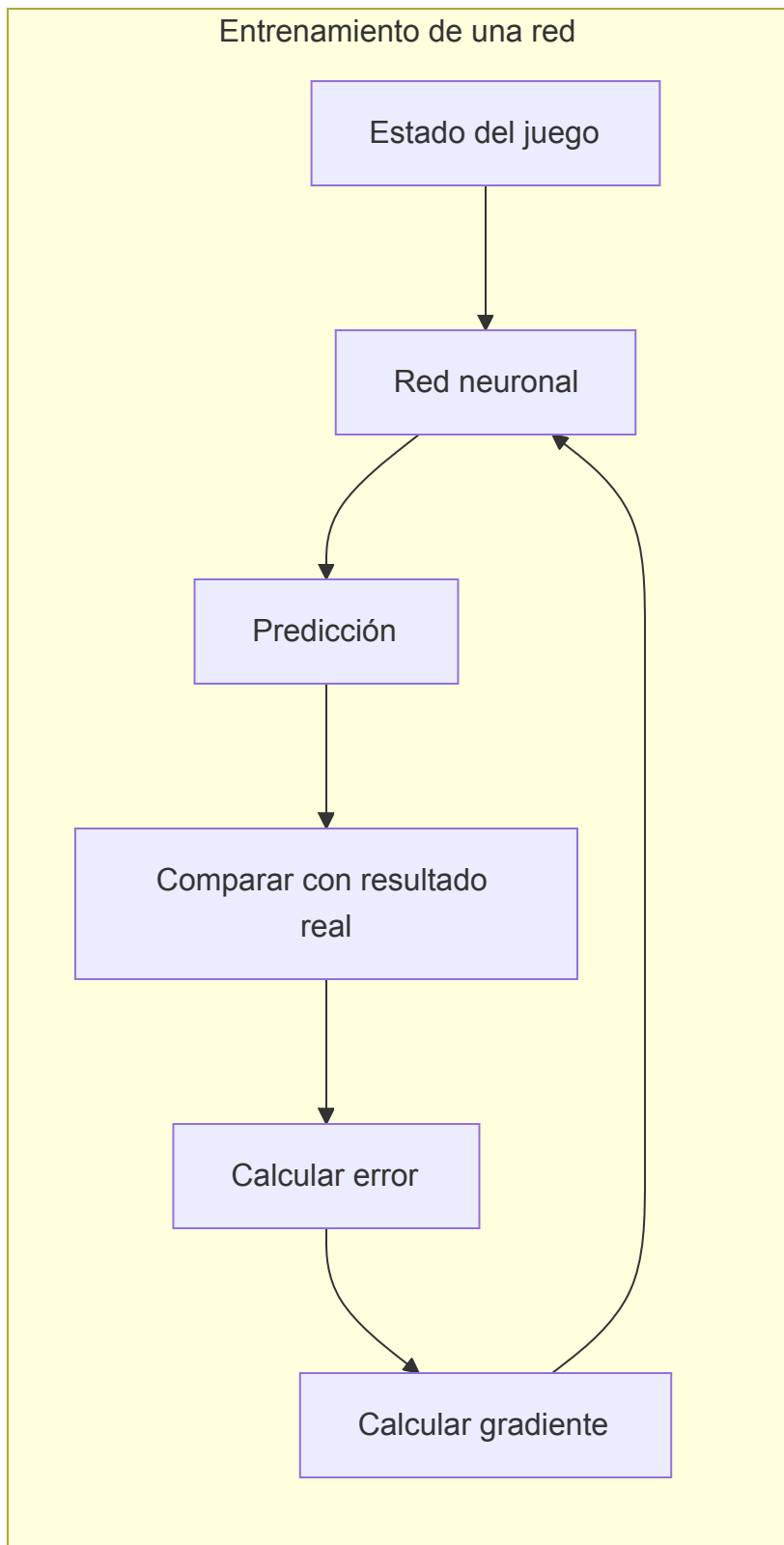
3.1 Derivadas

Intuición: La derivada mide "qué tan rápido cambia algo". Si estás manejando, la velocidad es la derivada de la posición.



3.2 El Gradiente

Intuición: El gradiente es la "dirección de máxima pendiente" en un mapa multidimensional. Es como pararte en una montaña y preguntar "¿hacia dónde es más empinado bajar?"



El loop de entrenamiento en una línea:

```
pesos_nuevos = pesos_viejos - learning_rate × gradiente
```

Donde `learning_rate` es qué tan rápido querés aprender (muy rápido → inestable, muy lento → no termina más).

3.3 Chain Rule

Intuición: Si A afecta a B y B afecta a C, entonces para cambiar C necesitás saber cómo A afecta a B primero.

En una red de 10 capas, el error en la capa 10 necesita "propagarse hacia atrás" hasta la capa 1. Eso es **backpropagation**, y usa la chain rule.

Relevancia para entrenar tu TrucoAI:

Cuando tu agente juega mal una carta, el error se propaga desde la decisión final hasta cada peso en la red. La chain rule es lo que hace posible ese cálculo.

Resumen de Fórmulas que REALMENTE vas a usar

Fórmula	Para qué	Ejemplo
$P(A) = \text{favorables} / \text{totales}$	Probabilidad base	¿Qué probabilidad de tener flor?
$P(A B) = P(A \cap B) / P(B)$	Probabilidad condicional	¿Prob de que tenga flor si cantó envido?
$E[X] = \sum x \cdot P(x)$	Valor esperado	¿Convierto el truco o no?
$y = W \cdot x + b$	Una neurona	Calcular valor de una acción
$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla L$	Gradient descent	Entrenar la red

Ejercicios prácticos

Ejercicio 1 — Simulación Monte Carlo (Truco)

Problema: Agarrá un mazo de 40 cartas. Repartí 3 cartas al azar. ¿Qué probabilidad hay de tener flor en la mano (3 cartas del mismo palo)?

```
import random

def simular_flor(n_simulaciones=100000):
    palos = ['E', 'B', 'O', 'C']
    valores = [1,2,3,4,5,6,7,10,11,12]
    mazo = [(v,p) for v in valores for p in palos]
```

```

flores = 0
for _ in range(n_simulaciones):
    mano = random.sample(mazo, 3)
    palos_mano = [carta[1] for carta in mano]
    if palos_mano[0] == palos_mano[1] == palos_mano[2]:
        flores += 1

return flores / n_simulaciones

prob = simular_flor()
print(f"Probabilidad de flor ≈ {prob*100:.2f}%")
# Resultado esperado: ~4.8-5.2%

```

Ejercicio 2 — Valor Esperado de un Truco

Problema: Estás 12-12 en la final. Tenés mano regular. El oponente canta Truco.

- Estimás que ganás la mano con 40% de probabilidad
- Si aceptás, el juego vale 3 puntos
- Si te vas al mazo, perdés 1 punto

¿Aceptás o te vas?

```

E[aceptar] = 0.4 × 3 + 0.6 × (-3) = -0.6
E[vete] = -1

```

→ Aceptar es mejor ($-0.6 > -1$)

Ejercicio 3 — Dot Product en Python

```

import numpy as np

# Estado del juego simplificado para Truco
estado = np.array([3.0, 2.0, 7.0, # tus cartas
                  1.0, 0.0, 0.0, # ronda actual jugadas
                  12.0, 5.0,     # scores
                  1.0])          # truco activo (sí/no)

# Pesos de la red (simplificado)
pesos = np.random.randn(9, 4) # 9 entradas, 4 acciones posibles

# Qué tan buena es cada acción
acciones = pesos.T @ estado # dot product
print(f"Valor de cada acción: {acciones}")

```

```
# acción 0=Jugar carta baja, 1=Jugar carta media, 2=Jugar carta alta,  
3=Cantar truco
```

Recursos adicionales

- **Video:** 3Blue1Brown — "Essence of Linear Algebra" (YouTube, serie completa de 16 videos, 10 min c/u)
 - **Video:** 3Blue1Brown — "Probability" playlist
 - **Video:** StatQuest — Probability Fundamentals
 - **Libro:** "Mathematics for Machine Learning" — Deisenroth, Faisal, Ong (gratuito online)
 - **Interactive:** Seeing Theory (seeingtheory.brown.edu) — visualizaciones interactivas de probabilidad
-

Checkpoint

Antes de pasar a la Lección 0.2, deberías poder:

- ☐ Explicar probabilidad condicional usando un ejemplo de Truco
 - ☐ Calcular el valor esperado de aceptar o rechazar un Truco
 - ☐ Saber qué es un vector y cómo representa el estado de un juego
 - ☐ Entender que una red neuronal = matrices $\times$ vectores + bias
 - ☐ Explicar qué hace el gradiente durante el entrenamiento
-

Siguiente: [Lección 0.2 — Python, NumPy, y PyTorch](#)